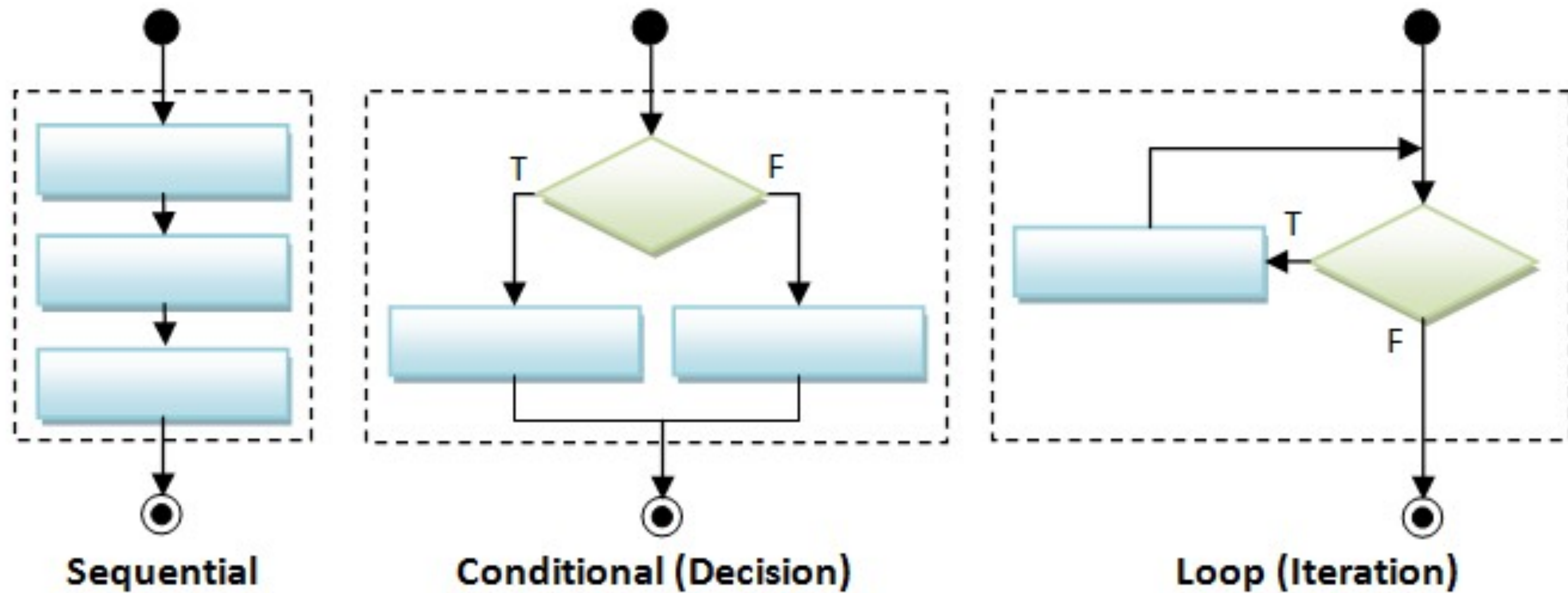


CONTROL FLOW



A Control flow is the way a program decides which instructions to execute next based on conditions or repetition.



You use Control Flow everywhere

- Check user input
- Handle errors
- CLI tools
- ...

Control flow in Rust

Rust Control Flow Summary

Type	Syntax Example	Description
<code>if / else</code>	<code>if x > 0 { ... } else { ... }</code>	Conditional branching
<code>loop</code>	<code>loop { ... }</code>	Infinite loop; must use <code>break</code> to exit
<code>while</code>	<code>while x > 0 { ... }</code>	Repeats while a condition is true
<code>for</code>	<code>for i in 0..3 { ... }</code>	Iterates over a range or collection
<code>match</code>	<code>match x { 1 => ..., _ => ... }</code>	Pattern matching; handles multiple conditions
<code>if let</code>	<code>if let Some(x) = opt { ... }</code>	Matches a specific pattern, simpler than <code>match</code>
<code>while let</code>	<code>while let Some(x) = opt { ... }</code>	Loops while a pattern keeps matching

1. If / else



```
fn main() {  
    let number = 7;  
  
    if number > 0 {  
        println!("The number is positive.");  
    } else {  
        println!("The number is zero or negative.");  
    }  
}
```

1. If / else



```
fn main() {  
    let number = 7;  
    println!("{}", if number > 0 { "Positive" } else { "Not positive" });  
}
```


2. Loop



```
fn main() {  
    let mut count = 0;  
  
    loop {  
        println!("Count is: {}", count);  
        count += 1;  
  
        if count == 3 {  
            break;  
        }  
    }  
}
```

3. While



```
fn main() {  
    let mut number = 3;  
  
    while number > 0 {  
        println!("Number: {}", number);  
        number -= 1;  
    }  
  
    println!("Done!");  
}
```

4. For



```
fn main() {  
    for i in 1..4 {  
        println!("i is: {}", i);  
    }  
}
```


5. Match



```
fn main() {  
    let number = 2;  
  
    match number {  
        1 => println!("One"),  
        2 => println!("Two"),  
        3 => println!("Three"),  
        _ => println!("Something else"),  
    }  
}
```

6. && and ||

```
fn main() {  
    let age = 20;  
    let has_id = true;  
  
    if age >= 18 && has_id {  
        println!("Access granted.");  
    }  
  
    let is_admin = false;  
    let is_logged_in = true;  
  
    if is_admin || is_logged_in {  
        println!("Welcome to your dashboard.");  
    }  
}
```

Later, we will deep diver in **iterations** in Rust.

Let's code!

Quick recap

If / else

Loop

While

For

Match

&&

Your turn!

